

# Project1: 仿真环境搭建与动力学验证作业说明

## 1. 项目目标

验证浮动基座动力学的物理正确性，建立 "数学模型 → 仿真验证" 的闭环工程直觉。

通过一个最简单但最根本的物理 sanity check：在没有任何外力和地面约束的情况下，四足机器人的质心应当做标准自由落体运动，其加速度恒为  $\mathbf{a} = [0, 0, -9.81]^\top \text{ m/s}^2$ 。

你将学到：

- MuJoCo 仿真引擎的基本使用流程
- 多刚体系统广义质量矩阵  $\mathbf{M}(\mathbf{q})$  和重力向量  $\mathbf{g}(\mathbf{q})$  的物理含义与计算方法
- 如何通过数值微分从位置数据中提取加速度
- 如何定量评估仿真结果的物理正确性

---

## 2. 背景知识

### 2.1 浮动基座动力学方程

对于一个具有浮动基座的多刚体系统（如四足机器人），其运动方程为：

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{g} = \mathbf{S}^\top \boldsymbol{\tau} + \sum_i \mathbf{J}_{c,i}^\top \mathbf{f}_c^{(i)}$$

各符号含义：

| 符号                           | 含义                                                                       | 维度               |
|------------------------------|--------------------------------------------------------------------------|------------------|
| $\mathbf{q}$                 | 广义坐标（基座位姿 + 关节角度）                                                        | $n_q$            |
| $\dot{\mathbf{q}}$           | 广义速度                                                                     | $n_v$            |
| $\mathbf{M}(\mathbf{q})$     | 广义质量矩阵（惯性矩阵），对称正定                                                        | $n_v \times n_v$ |
| $\mathbf{C}\dot{\mathbf{q}}$ | 科里奥利力 + 离心力                                                              | $n_v$            |
| $\mathbf{g}(\mathbf{q})$     | 重力项                                                                      | $n_v$            |
| $\mathbf{S}$                 | 选择矩阵 $[\mathbf{0}_{n_a \times 6} \quad \mathbf{I}_{n_a}]$ ，将关节力矩映射到广义力空间 | $n_a \times n_v$ |
| $\boldsymbol{\tau}$          | 关节驱动力矩                                                                   | $n_a$            |
| $\mathbf{J}_{c,i}$           | 第 $i$ 个接触点的雅可比矩阵                                                         | $3 \times n_v$   |
| $\mathbf{f}_c^{(i)}$         | 第 $i$ 个接触点的接触力                                                           | 3                |

**关键理解：** $\mathbf{S}^\top \boldsymbol{\tau}$  中的选择矩阵  $\mathbf{S}$  体现了浮动基座系统的核心特征——关节力矩 **只能作用于关节自由度**（后  $n_a = 12$  维），**无法直接驱动基座的 6 个自由度**（前 6 维）。基座的运动完全依赖接触力  $\mathbf{f}_c$  间接驱动。这也是为什么失去地面接触时基座不可控。

## 2.2 自由落体的物理推导

当  $\boldsymbol{\tau} = \mathbf{0}$ （无关节力矩）且  $\mathbf{f}_c^{(i)} = \mathbf{0}$ （无接触力/无地面）时，运动方程简化为：

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{g} = \mathbf{0}$$

对于 **质心运动**，由牛顿第二定律可知：

$$m_{\text{total}} \cdot \ddot{\mathbf{p}}_{\text{CoM}} = m_{\text{total}} \cdot \mathbf{g}_0$$

其中  $\mathbf{g}_0 = [0, 0, -9.81]^\top$  为重力加速度向量。因此基座的加速度应恒为：

$\ddot{\mathbf{p}}_B \approx [0, 0, -9.81]^\top \text{ m/s}^2$

**注意：**基座并不严格等于质心，因为四肢关节在重力下可能会有微小的相对运动，但由于基座质量远大于四肢（约占总质量的 50%+），且初始速度为零，短时间内基座加速度与质心加速度高度一致。

## 2.3 广义质量矩阵的性质

物理正确的广义质量矩阵  $\mathbf{M}(\mathbf{q})$  必须满足：

- 1. **对称性：**  $\mathbf{M} = \mathbf{M}^\top$  （因为动能是  $\dot{\mathbf{q}}$  的二次型）
- 2. **正定性：**  $\mathbf{x}^\top \mathbf{M} \mathbf{x} > 0$  ，对任意非零向量  $\mathbf{x}$  （因为任何非零速度都对应正的动能）

验证这两个性质是确认模型参数（质量、惯性张量）正确的第一步。

## 3. 项目文件结构

拿到本项目文件夹后，你会看到以下文件结构。请先确认文件完整：

代码块

```
1  project_1_freefall_verification/
2  |
3  |-- Project1_作业说明.pdf          <-- 本文档（教学指导）
4  |-- freefall_verification.py      <-- 完整参考代码（可直接运行）
5  |-- convert_urdf_to_mjcf.py       <-- URDF 转 MJCF 演示脚本
6  |-- requirements.txt              <-- Python 依赖列表
7  |-- setup_env.sh                  <-- 一键环境配置脚本（可选）
8  |-- freefall_results.png          <-- 运行后生成的结果图表
9  |-- freefall_animation.gif        <-- 运行后生成的自由落体 3D 动画
10 |
11 |-- resources/xg/
12 |   |-- urdf/
13 |     |-- xg.urdf                 <-- XG 四足机器人原始 URDF 模型
14 |     |-- xg.csv                   <-- 关节参数表
```

```

15 | | +-- *.STL <-- mesh 文件副本 (MuJoCo 加载 URDF 需
    | | 要)
16 | |
17 | | |-- meshes/
18 | | |-- BASE_LINK.STL <-- 机器人各部件的 3D 模型文件
19 | | |-- FL_ABAD_LINK.STL
20 | | |-- ... (共 17 个 STL 文件)
21 | | +-- RR_FOOT_LINK.STL
22 | |
23 | | |-- xg_freefall.xml <-- 自由落体验证专用 MJCF (无地面、简化碰
    | | 撞体)
24 | | +-- xg_converted.xml <-- 完整 MJCF (含地面、mesh 渲染、碰撞体)

```

## 说明:

- `xg.urdf` 是原始机器人模型 (SolidWorks 导出), 用于学习 URDF → MJCF 转换流程
- `xg_freefall.xml` 是专门为自由落体验证准备的简化模型 (无地面、无关节阻尼、无 mesh 依赖)
- `xg_converted.xml` 是从 URDF 转换并优化后的完整 MJCF 模型 (含地面和 mesh 渲染)
- `meshes/` 中包含机器人各部件的 STL 3D 模型文件, 被 `xg_converted.xml` 引用
- `urdf/` 目录下也包含了 STL 文件的副本, 这是因为 MuJoCo 加载 URDF 时要求 mesh 文件与 URDF 在同一目录

## 4. 环境配置

### 4.1 系统要求

- 操作系统: **推荐** Ubuntu 20.04 或更高版本, 经测试 macOS、Windows、WSL 也可以运行
- Python: 3.8 或更高版本 (推荐 3.10)
- GPU: **不需要**, MuJoCo CPU 版本即可完成本项目

## 4.2 方式一：一键配置

在终端中进入项目目录，运行配置脚本：

代码块

```
1 cd project_1_freefall_verification
2 bash setup_env.sh
```

脚本会自动创建 conda 环境 `freefall` 并安装所有依赖。

## 4.3 方式二：手动配置

如果一键脚本报错，可以手动完成以下步骤。

### Step 1: 安装 Conda（如果已安装则跳过）

代码块

```
1 mkdir -p ~/miniconda3
2 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
~/miniconda3/miniconda.sh
3 bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
4 rm ~/miniconda3/miniconda.sh
5 ~/miniconda3/bin/conda init --all
6 source ~/.bashrc
```

### Step 2: 创建虚拟环境

代码块

```
1 conda create -n freefall python=3.10 -y
```

```
2 conda activate freefall
```

### Step 3: 安装 Python 依赖

代码块

```
1 cd project_1_freefall_verification
2 pip install -r requirements.txt
```

requirements.txt 的内容如下：

代码块

```
1 mujoco>=3.0.0
2 numpy>=1.24.0
3 matplotlib>=3.7.0
4 Pillow>=9.0.0
```

### Step 4: 验证安装

代码块

```
1 python -c "import mujoco; print(f'mujoco {mujoco.__version__} OK')"
2 python -c "import numpy; print(f'numpy {numpy.__version__} OK')"
3 python -c "import matplotlib; print(f'matplotlib {matplotlib.__version__} OK')"
```

三行都打印 OK 即表示环境就绪。

## 4.4 快速运行验证

环境配置完成后，直接运行参考代码验证一切正常：

代码块

```
1  conda activate freefall
2  cd project_1_freefall_verification
3  python freefall_verification.py
```

如果看到最后输出 "自由落体验证: PASS"，说明环境和文件都没有问题。

## 5. 核心任务流程

### 5.1 快速开始（3 步完成全部代码运行）

本项目的核心代码已经整合在 `freefall_verification.py` 中。你只需以下 3 步即可完成全部代码部分的作业：

代码块

```
1  cd project_1_freefall_verification
2  bash setup_env.sh
3  conda activate freefall
4  python freefall_verification.py
```

运行完成后，脚本会自动执行全部 4 个 Task 并输出所有验证结果，同时在当前目录下生成：

- `freefall_results.png` — 数据分析图表（含位置、速度、加速度、位移 4 张子图）
- `freefall_animation.gif` — XG 四足机器人自由落体 3D 动画

**你还需要完成的工作：**根据代码运行结果，撰写一份不超过 3 页的分析报告（详见第 8 节"成果交付与考核标准"）。**评分重点不是"把代码跑通"，而是"能否合理解释每一步数值的物理含义"。**

本文档后续章节（第 4-7 节）将逐步讲解 `freefall_verification.py` 中每一段代码的原理和物理含义，帮助你理解代码并撰写分析报告。

**说明：**以下代码块均来自 `freefall_verification.py`，是对该脚本的逐步分解讲解。你 **不需要** 手动逐段复制运行这些代码，直接运行 `python freefall_verification.py` 即可一次性执行全部任务。目的是帮助你 **理解每一步的原理和物理含义**，为撰写分析报告做准备。

## 5.1 Task 1：加载模型

### 5.1.1 理解 URDF 与 MJCF

| 格式   | 全称                               | 特点                     |
|------|----------------------------------|------------------------|
| URDF | Unified Robot Description Format | ROS 生态标准，树形结构          |
| MJCF | MuJoCo XML Format                | MuJoCo 原生格式，支持更丰富的物理参数 |

本项目使用的 XG 四足机器人模型原始格式为 URDF（由 SolidWorks 导出），文件位于：

代码块

```
1 resources/xg/urdf/xg.urdf
```

### 5.1.2 URDF 转 MJCF



MuJoCo 提供了内置的编译工具，可以直接将 URDF 转换为 MJCF。

本项目提供了一个转换演示脚本，可以直接运行：

代码块

```
1 cd project_1_freefall_verification
2 python convert_urdf_to_mjcf.py
```

运行后会在 resources/xg/ 目录下生成 xg\_from\_urdf.xml 文件。

### 重要提示：

1. MuJoCo 加载 URDF 时，要求 mesh 文件（STL）与 URDF 文件在同一目录下，所以 urdf/ 目录中包含了 STL 文件的副本
2. URDF 默认是固定基座模型（nv=12），没有浮动基座的 6 个自由度。要模拟自由落体，需要在 MJCF 中手动添加 freejoint 标签

在本项目中，我们已经预先完成了 URDF 到 MJCF 的转换，并做了以下优化：

- 添加了 freejoint（浮动基座，nv 从 12 变为 18）
- 将 mesh 碰撞体替换为球体碰撞体（提高仿真速度）
- 调整了接触参数和关节阻尼

优化后的文件有两个版本：

- resources/xg/xg\_freefall.xml -- 自由落体专用（无地面、无关节阻尼）
- resources/xg/xg\_converted.xml -- 完整版（含地面、含 mesh 渲染）

### 5.1.3 加载 MJCF 模型

#### 代码块

```
1 import mujoco
2 import numpy as np
3
4 model_path = "resources/xg/xg_freefall.xml"
5 model = mujoco.MjModel.from_xml_path(model_path)
6 data = mujoco.MjData(model)
7
8 print(f"自由度 (nv): {model.nv}")
9 print(f"广义坐标维度 (nq): {model.nq}")
10 print(f"关节数: {model.njnt}")
11 print(f"刚体数: {model.nbody}")
12 print(f"执行器数: {model.nu}")
13 print(f"仿真步长: {model.opt.timestep} s")
14
15 total_mass = sum(model.body_mass)
16 print(f"系统总质量: {total_mass:.4f} kg")
```

#### 预期输出:

#### 代码块

```
1 自由度 (nv): 18
2 广义坐标维度 (nq): 19
3 关节数: 13
4 刚体数: 18
5 执行器数: 12
6 仿真步长: 0.001 s
7 系统总质量: 14.9734 kg
```

**关键概念:** 为什么  $nq=19$  但  $nv=18$ ? 因为基座的朝向用 4 维四元数表示 ( $nq$  中占 4 维), 但其角速度只有 3 个自由度 ( $nv$  中占 3 维)。

---

## 5.2 Task 2：计算静态量

### 5.2.1 选定标准站立姿态 $q_0$

XG 四足机器人的标准站立姿态：

| 关节类型           | 角度 (rad) | 说明     |
|----------------|----------|--------|
| ABAD（髋关节外展/内收） | 0.0      | 腿部不外展  |
| HIP（髋关节屈/伸）    | 0.8      | 大腿向后弯曲 |
| KNEE（膝关节）      | -1.5     | 小腿向前弯曲 |

代码块

```
1  q0 = np.zeros(model.nq)
2  q0[0:3] = [0.0, 0.0, 0.4]           # 基座位置 (x, y, z)
3  q0[3:7] = [1.0, 0.0, 0.0, 0.0]     # 基座朝向 (四元数 w,x,y,z)
4
5  # 12 个关节角度：[ABAD, HIP, KNEE] x 4条腿
6  # 关节顺序 (在 MJCF 中的定义顺序):
7  # FAR(前右): ABAD, HIP, KNEE
8  # FBL(前左): ABAD, HIP, KNEE
9  # RAR(后右): ABAD, HIP, KNEE
10 # RBL(后左): ABAD, HIP, KNEE
11 joint_angles = np.array([
12     0.0, 0.8, -1.5,  # FAR (Front Right)
13     0.0, 0.8, -1.5,  # FBL (Front Left)
14     0.0, 0.8, -1.5,  # RAR (Rear Right)
15     0.0, 0.8, -1.5,  # RBL (Rear Left)
16 ])
17 q0[7:19] = joint_angles
18
19 # 设置广义坐标并执行前向运动学
20 data.qpos[:] = q0
21 data.qvel[:] = 0    # 初始速度为零
22 mujoco.mj_forward(model, data)
```

### 5.2.2 计算广义质量矩阵 $M(q_0)$

MuJoCo 将质量矩阵以稀疏格式存储在 `data.qM` 中，可以用 `mj_fullM` 函数展开为稠密矩阵：

代码块

```
1 M = np.zeros((model.nv, model.nv))
2 mujoco.mj_fullM(model, M, data.qM)
3
4 print(f"质量矩阵 M(q0) 的形状: {M.shape}")
5 print(f"质量矩阵的条件数: {np.linalg.cond(M):.2f}")
```

### 5.2.3 验证 $M(q_0)$ 的对称正定性

代码块

```
1 # ---- 验证对称性 ----
2 symmetry_error = np.max(np.abs(M - M.T))
3 is_symmetric = symmetry_error < 1e-10
4 print(f"对称性检验: max|M - M^T| = {symmetry_error:.2e}")
5 print(f"是否对称: {is_symmetric}")
6
7 # ---- 验证正定性 ----
8 eigenvalues = np.linalg.eigvalsh(M)
9 is_positive_definite = np.all(eigenvalues > 0)
10 print(f"最小特征值: {eigenvalues[0]:.6f}")
11 print(f"最大特征值: {eigenvalues[-1]:.6f}")
12 print(f"是否正定: {is_positive_definite}")
13
14 # ---- 物理解读 ----
15 total_mass = sum(model.body_mass)
16 print(f"基座平移惯量 M[0,0] = {M[0,0]:.4f} kg")
17 print(f"模型总质量 = {total_mass:.4f} kg")
18 print(f"两者是否一致: {abs(M[0,0] - total_mass) < 0.01}")
```

预期结果：

- 对称性误差  $< 10^{-10}$  : PASS
- 所有特征值  $> 0$  : PASS
- $M[0,0] \approx$  模型总质量 14.97 kg : PASS

**物理含义：**质量矩阵对角线的前 3 个元素对应基座平移惯量，近似等于系统总质量；接下来的 3 个对应基座旋转惯量；剩余 12 个对应各关节的等效惯量。

#### 5.2.4 计算重力向量 $\mathbf{g}(\mathbf{q}_0)$

在 MuJoCo 中，`data.qfrc_bias` 存储的是  $\mathbf{C}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$ 。当速度为零时，科里奥利力项  $\mathbf{C}\dot{\mathbf{q}}$  为零，因此 `qfrc_bias` 就等于纯重力项：

代码块

```
1 data.qvel[:] = 0
2 mujoco.mj_forward(model, data)
3
4 g_vec = data.qfrc_bias.copy()
5
6 print(f"重力向量 g(q0) 的形状: {g_vec.shape}")
7 print(f"基座线性分量: g[0:3] = {g_vec[0:3]}")
8 print(f"基座角度分量: g[3:6] = {g_vec[3:6]}")
9
10 # 物理解读:
11 # MuJoCo 运动方程: M*qacc + qfrc_bias = tau
12 # 所以 z 方向重力偏置力为正值 (+mg)
13 print(f"g[2] (z方向) = {g_vec[2]:.4f} N")
14 print(f"理论值 +m*g = {total_mass * 9.81:.4f} N")
```

**预期结果：**  $g[2] \approx +m_{\text{total}} \times 9.81 = 146.89 \text{ N}$

### 5.3 Task 3: 自由落体仿真

### 5.3.1 准备无地面模型

为了模拟自由落体，需要移除地面约束。本项目提供了两种方式：

方式 1 -- 使用预置的无地面模型（推荐）：

代码块

```
1 model = mujoco.MjModel.from_xml_path("resources/xg/xg_freefall.xml")
```

方式 2 -- 在代码中手动禁用碰撞：

代码块

```
1 model = mujoco.MjModel.from_xml_path("resources/xg/xg_converted.xml")
2 for i in range(model.ngeom):
3     model.geom_contype[i] = 0
4     model.geom_conaffinity[i] = 0
```

### 5.3.2 执行自由落体仿真

代码块

```
1 # 重新初始化到标准姿态
2 data.qpos[:] = q0.copy()
3 data.qvel[:] = 0
4 mujoco.mj_forward(model, data)
5
6 # 仿真参数
7 dt = 0.001          # 步长 1 ms
8 num_steps = 100     # 100 步 -> 总时长 0.1 s
9
10 # 设置模型步长
```

```

11  model.opt.timestep = dt
12
13  # 数据记录数组
14  time_log = np.zeros(num_steps + 1)
15  pos_log = np.zeros((num_steps + 1, 3))
16  vel_log = np.zeros((num_steps + 1, 3))
17
18  # 记录初始状态
19  time_log[0] = 0.0
20  pos_log[0] = data.qpos[0:3].copy()
21  vel_log[0] = data.qvel[0:3].copy()
22
23  print(f"步长: {dt} s, 步数: {num_steps}, 总时长: {dt * num_steps} s")
24  print(f"初始位置: {pos_log[0]}")
25
26  for step in range(num_steps):
27      # 关节力矩设为零
28      data.ctrl[:] = 0
29      data.qfrc_applied[:] = 0
30
31      # 执行一步仿真
32      mujoco.mj_step(model, data)
33
34      # 记录数据
35      time_log[step + 1] = (step + 1) * dt
36      pos_log[step + 1] = data.qpos[0:3].copy()
37      vel_log[step + 1] = data.qvel[0:3].copy()
38
39  print(f"仿真完成! 最终位置: {pos_log[-1]}")
40  print(f"z 方向下落距离: {pos_log[0, 2] - pos_log[-1, 2]:.6f} m")
41  print(f"理论下落距离: {0.5 * 9.81 * (dt * num_steps)**2:.6f} m")

```

## 5.4 Task 4: 物理特征验证

### 5.4.1 计算基座加速度

使用中心差分法从速度数据中提取加速度:

$$\ddot{\mathbf{p}}_B(t_i) \approx \frac{\dot{\mathbf{p}}_B(t_{i+1}) - \dot{\mathbf{p}}_B(t_{i-1})}{2 \Delta t}$$

代码块

```
1  acc_log = np.zeros((num_steps - 1, 3))
2  for i in range(1, num_steps):
3      acc_log[i - 1] = (vel_log[i + 1] - vel_log[i - 1]) / (2 * dt)
4
5  print("=== 加速度验证结果 ===")
6  print(f"理论加速度: [0, 0, -9.81] m/s^2")
7  print(f"平均 a_x = {np.mean(acc_log[:, 0]):.6f} m/s^2 (期望: 0)")
8  print(f"平均 a_y = {np.mean(acc_log[:, 1]):.6f} m/s^2 (期望: 0)")
9  print(f"平均 a_z = {np.mean(acc_log[:, 2]):.6f} m/s^2 (期望: -9.81)")
10
11 # 计算相对误差
12 az_mean = np.mean(acc_log[:, 2])
13 relative_error = abs(az_mean - (-9.81)) / 9.81 * 100
14 print(f"z 方向相对误差: {relative_error:.4f}%")
15 print(f"是否满足 < 1%: {'PASS' if relative_error < 1 else 'FAIL'}")
```

## 5.4.2 对比理论轨迹

代码块

```
1  z_theory = pos_log[0, 2] - 0.5 * 9.81 * time_log**2
2  vz_theory = -9.81 * time_log
3
4  pos_error = np.abs(pos_log[:, 2] - z_theory)
5  vel_error = np.abs(vel_log[:, 2] - vz_theory)
6  print(f"最大位置误差: {np.max(pos_error):.8f} m")
7  print(f"最大速度误差: {np.max(vel_error):.8f} m/s")
```

## 5.4.3 绘制结果图表

代码块

```
1  import matplotlib.pyplot as plt
```



```

2
3 fig, axes = plt.subplots(2, 2, figsize=(12, 9))
4 fig.suptitle("Project 1: Free-Fall Verification", fontsize=14,
5 fontweight='bold')
6
7
8 # (a) z 方向位置
9 ax = axes[0, 0]
10 ax.plot(t_ms, pos_log[:, 2], 'b-', linewidth=2, label='Simulation')
11 ax.plot(t_ms, z_theory, 'r--', linewidth=1.5, label='Theory')
12 ax.set_xlabel('Time (ms)')
13 ax.set_ylabel('z position (m)')
14 ax.set_title('(a) Base z-position')
15 ax.legend()
16 ax.grid(True, alpha=0.3)
17
18 # (b) z 方向速度
19 ax = axes[0, 1]
20 ax.plot(t_ms, vel_log[:, 2], 'b-', linewidth=2, label='Simulation')
21 ax.plot(t_ms, vz_theory, 'r--', linewidth=1.5, label='Theory: -gt')
22 ax.set_xlabel('Time (ms)')
23 ax.set_ylabel('z velocity (m/s)')
24 ax.set_title('(b) Base z-velocity')
25 ax.legend()
26 ax.grid(True, alpha=0.3)
27
28 # (c) z 方向加速度
29 ax = axes[1, 0]
30 t_acc = time_log[1:-1] * 1000
31 ax.plot(t_acc, acc_log[:, 2], 'b-', linewidth=2, label='Simulation')
32 ax.axhline(y=-9.81, color='r', linestyle='--', linewidth=1.5, label='Theory: -
33 g')
34 ax.set_xlabel('Time (ms)')
35 ax.set_ylabel('z acceleration (m/s^2)')
36 ax.set_title('(c) Base z-acceleration')
37 ax.legend()
38 ax.grid(True, alpha=0.3)
39
40 # (d) 各轴位移
41 ax = axes[1, 1]
42 ax.plot(t_ms, pos_log[:, 0], '-', linewidth=2, label='x')
43 ax.plot(t_ms, pos_log[:, 1], '-', linewidth=2, label='y')
44 ax.plot(t_ms, pos_log[:, 2] - pos_log[0, 2], '-', linewidth=2, label='delta z')
45 ax.set_xlabel('Time (ms)')
46 ax.set_ylabel('Displacement (m)')
47 ax.set_title('(d) All-axis displacement')

```

```
47 ax.legend()
48 ax.grid(True, alpha=0.3)
49
50 plt.tight_layout()
51 plt.savefig('freefall_results.png', dpi=150, bbox_inches='tight')
52 plt.show()
53 print("图表已保存至 freefall_results.png")
```

## 6. 进阶探索（选做）

- 步长敏感性分析：**将步长分别设为 0.1ms、1ms、5ms、10ms，比较加速度误差，绘制误差-步长关系图。
- 不同姿态验证：**改变初始关节角度（如全零姿态），观察结果是否依然满足自由落体条件。
- 角动量守恒验证：**在自由落体过程中，系统总角动量应当守恒（无外力矩）。提取角速度数据并验证。
- 使用 MJX（JAX 加速）：**将仿真代码迁移到 MJX 后端，利用 JAX 的 jit 编译加速仿真，对比运行速度。

## 7. MuJoCo 关键 API 速查

| API                         | 功能              | 用法示例                                              |
|-----------------------------|-----------------|---------------------------------------------------|
| MjModel.from_xml_path(path) | 从 XML/URDF 加载模型 | model =<br>mujoco.MjModel.from_xml_path("xg.xml") |

|                             |               |                             |
|-----------------------------|---------------|-----------------------------|
| MjData(model)               | 创建仿真数据        | data = mujoco.MjData(model) |
| mj_forward(model, data)     | 前向运动学（不积分）    | 设置 qpos 后调用                 |
| mj_step(model, data)        | 一步仿真积分        | 包含碰撞检测和积分                   |
| mj_fullM(model, M, data.qM) | 提取完整质量矩阵      | M 为 (nv, nv) 数组             |
| data.qpos                   | 广义坐标          | 基座(7) + 关节(12) = 19         |
| data.qvel                   | 广义速度          | 基座(6) + 关节(12) = 18         |
| data.qfrc_bias              | 偏置力 (科里奥利+重力) | 速度为零时 = 重力项                 |
| data.ctrl                   | 执行器控制输入       | 12 维                        |
| model.opt.timestep          | 仿真步长          | 默认 0.002s                   |

## 8. XG 四足机器人模型说明

### 8.1 机器人结构

代码块

```
1  BASE_LINK (torso)  --- freejoint -- 世界坐标系
2  |-- FAR (Front Right)
3  |   |-- FAR_ABAD_JOINT  (外展/内收, x轴旋转)
4  |   |-- FAR_HIP_JOINT   (屈/伸, y轴旋转)
5  |   +--- FAR_KNEE_JOINT (膝关节, y轴旋转)
6  |-- FBL (Front Left)
7  |   |-- FBL_ABAD_JOINT
8  |   |-- FBL_HIP_JOINT
9  |   +--- FBL_KNEE_JOINT
10 |-- RAR (Rear Right)
11 |   |-- RAR_ABAD_JOINT
12 |   |-- RAR_HIP_JOINT
13 |   +--- RAR_KNEE_JOINT
14 +--- RBL (Rear Left)
15      |-- RBL_ABAD_JOINT
```

16 |-- RBL\_HIP\_JOINT

17 +-- RBL\_KNEE\_JOINT

## 8.2 关键参数

| 参数   | 值             | 说明                 |
|------|---------------|--------------------|
| 基座质量 | 6.27 kg       | BASE_LINK          |
| 总质量  | 约 15 kg       | 含 4 条腿             |
| 自由度  | 18 (6+12)     | 6 基座 + 12 关节       |
| 腿长   | 约 0.41 m      | 大腿 0.2m + 小腿 0.21m |
| 站立高度 | 约 0.35-0.40 m | 取决于关节角度            |

## 8.3 广义坐标排布

代码块

1 qpos[0:3] - 基座位置 (x, y, z)

2 qpos[3:7] - 基座四元数 (w, x, y, z)

3 qpos[7:10] - FAR: ABAD, HIP, KNEE

4 qpos[10:13] - FBL: ABAD, HIP, KNEE

5 qpos[13:16] - RAR: ABAD, HIP, KNEE

6 qpos[16:19] - RBL: ABAD, HIP, KNEE

7

8 qvel[0:3] - 基座线速度 (vx, vy, vz)

9 qvel[3:6] - 基座角速度 (wx, wy, wz)

10 qvel[6:18] - 12 个关节速度

## 9. 成果交付与考核标准

### 9.1 交付物

| 序号 | 内容        | 格式          | 说明                         |
|----|-----------|-------------|----------------------------|
| 1  | Python 脚本 | .py (约50行)  | 完整可运行的自由落体验证代码             |
| 2  | 位置时间历程曲线图 | .png        | 基座 z 轴位置随时间的变化曲线（仿真 vs 理论） |
| 3  | 分析报告      | .pdf (3页以内) | 含数值结果与误差源解释                |

### 9.2 分析报告要点

报告应包含但不限于以下内容：

#### 1. 质量矩阵分析

- $\mathbf{M}(\mathbf{q}_0)$  的对称性和正定性验证结果
- 对角线元素的物理含义解释
- 条件数分析（条件数过大意味着什么？）

#### 2. 自由落体结果分析

- 基座 z 轴位置时间历程曲线
- 加速度数值与理论值  $-9.81 \text{ m/s}^2$  的对比
- 相对误差量化

#### 3. 误差源讨论

- 数值积分误差（Euler 积分 vs 高阶方法）
- 时间步长对精度的影响
- 基座加速度与质心加速度的差异（四肢惯性效应）

- 关节阻尼/摩擦对结果的影响

## 9.3 评分重点

不仅是"把代码成功跑通", 而是"能否合理解释每一步数值的物理含义"。

例如:

- 为什么  $M[0, 0]$  等于总质量而不是基座质量?
  - 为什么  $\mathbf{g}(\mathbf{q})$  的前两个分量  $(x, y)$  不严格为零?
  - 加速度的数值微分精度受什么因素影响?
  - 如果把步长从 1ms 改为 10ms, 精度会怎么变化? 为什么?
-